TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS

A MAJOR PROJECT PROGRESS REPORT ON

IMPLEMENTING READ/WRITE OVERLAY FILESYSTEM SUPPORT IN A MICROKERNEL BASED OPERATING SYSTEM

BY

Rijan Karki (PUR078BCT067)
Saurav Khanal (PUR078BCT080)
Spandan Guragain (PUR078BCT086)
Sudesh Subedi (PUR078BCT088)

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PURWANCHAL CAMPUS
DHARAN, NEPAL

FEBRUARY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Mr. Manoj Kumar Guragai, Head of the Department, for his invaluable guidance and unwavering support throughout the preparation of this proposal. His insightful suggestions and constructive feedback have been instrumental in shaping our work, and we are truly appreciative of the time and effort he dedicated to our development. His leadership has not only provided us with direction but has also inspired us to pursue excellence in our academic endeavors.

We are also deeply thankful to our esteemed teachers and faculty members for their encouragement and thoughtful insights, which have significantly enriched this proposal. Their commitment to academic excellence and their willingness to share knowledge have served as a constant source of inspiration for us. The collective guidance and support we received from them have greatly contributed to the refinement and improvement of our work, and we are sincerely grateful for their contributions.

Rijan Karki PUR078BCT067

Saurav Khanal PUR078BCT080

Spandan Guragain PUR078BCT086

Sudesh Subedi PUR078BCT088

ABSTRACT

This report presents the implementation of an overlay filesystem (OverlayFS) for RedoxOS, a microkernel based operating system written in a memory safe language called Rust. Overlay filesystems compose multiple filesystem layers into a unified view, storing modifications separately from read-only base layers. This technology is essential for containerization platforms, live operating systems, and atomic package management. Our implementation integrates overlay functionality within Redox's unique scheme-based architecture, leveraging safety guarantees of Rust while maintaining the security and isolation properties of the microkernel design.

The implementation follows a delegate pattern separating platform-independent overlay logic from RedoxOS-specific integration code, enabling comprehensive cross-platform testing. As of this midterm evaluation, we have successfully implemented essential filesystem operations including file read/write with copy-on-write semantics, directory merging across layers, whiteout-based deletion handling, and metadata management. The system has been tested within RedoxOS and demonstrates correct integration with standard utilities.

Keywords: *overlay filesystem, union filesystem, RedoxOS, microkernel, copy-on-write, containerization, scheme-based I/O, system calls*

CONTENTS

ACKNOWLEDGEMENT ii

ABSTRACT iii

LIST OF FIGURES vii

LIST OF ABBREVIATIONS viii

1 INTRODUCTION 1

1.1 Background . . . 1

1.2 Gap Identification . . . 1

1.3 Motivation . . . 2

1.4 Objectives . . . 2

2 RELATED THEORY 3

2.1 Redox OS . . . 3

2.1.1 Introduction . . . 3

2.2 Design . . . 3

2.2.1 Microkernel . . . 3

2.2.1.1 Advantages of Microkernel Architecture . . . 4

2.2.1.2 Disadvantages of Microkernel Architecture . . . 4

2.2.2 Schemes and Resouces . . . 4

2.3 Implementation of a Scheme . . . 5

2.3.1 Root Scheme . . . 6

2.3.2 Event Scheme . . . 6

2.3.3 Providing a Scheme . . . 6

2.3.4 Action of the Kernel . . . 7

2.4 Linux's OverlayFS . . . 8

2.4.1 Copy Up Operation . . . 9

2.4.2 Whiteouts and Opaque Directories . . . 9

2.4.3 Readdir and Renaming . . . 10

2.4.4 Permission Model . . . 10

3 LITERATURE REVIEW 11

4 METHODOLOGY 14

4.1 Development Approach ... 14

4.2 Requirement Analysis and Designing the Filesystem ... 15

4.2.1 Study of the Scheme Based Architecture ... 15

4.2.2 Overlay Filesystem Analysis ... 16

4.3 Implementation ... 16

4.4 Testing ... 17

4.4.1 Unit Testing ... 17

4.4.2 Functional Testing ... 17

4.4.3 Compatibility Testing ... 17

4.5 Project Timeline ... 18

5 IMPLEMENTATION 20

5.1 Overall Architecture ... 20

5.2 Core Data Structures ... 20

5.3 Path Resolution ... 21

5.4 Copy-On-Write Implementation ... 22

5.5 Whiteout Implementation ... 22

5.6 Directory Merge Algorithm ... 23

6 RESULTS 24

6.1 Implemented System ... 24

6.2 File Handle Management ... 25

6.3 Implemented System Calls ... 25

6.3.1 open, openat ... 25

6.3.2 read and write ... 26

6.3.3 close ... 26

6.3.4 getdents ... 26

6.3.5 unlink and rmdir ... 27

6.3.6 fstat, fstatvfs, fsize ... 28

6.3.7 fchmod and fchown ... 28

6.3.8 ftruncate and futimens ... 28

6.3.9 flink 28
6.3.10 fcntl, fevent, fpath 28
6.4 Copy-On-Write Behaviour 29
6.5 Whiteout Handling 31
6.6 Opaque Directories Handling 32
6.7 Testing 33
6.7.1 Unit Tests 33
6.7.2 Integration Tests 33
6.8 Identified Issues and Limitations 34
7 CONCLUSION AND FUTURE WORK 35
7.1 Conclusion 35
7.2 Future Work 35
REFERENCES 38

LIST OF FIGURES

Figure 1 Visual Representation of Layers in an Overlay Filesystem 9
Figure 2 Prototype Model 15
Figure 3 Project Timeline 19
Figure 4 Client process call hierarchy 20
Figure 5 Architecture diagram 25
Listing 1 Merged/Overlayed view as seen by the `getdents` syscall. 27
Listing 2 Actual Underlying Filesystem 27
Listing 3 COW as seen by the `getdents` syscall. 30
Listing 4 Actual Underlying Filesystem after COW 30
Listing 5 Removing of File as seen by the `getdents` syscall. 31
Listing 6 Actual Underlying Filesystem after Deletion of File 31
Listing 7 Removing Directory as seen by the `getdents` syscall. 32
Listing 8 Actual Underlying Filesystem after Removing Directory 32

LIST OF ABBREVIATIONS

BSD Berkeley Software Distribution
CD Compact Disc
CLI Command Line Interface
COW Copy On Write
FAT File Allocation Table
I/O Input/Output
IPC Inter-Process Communication
MVP Minimum Viable Product
OS Operating System
ROM Read-Only Memory
RAM Random Access Memory
USB Universal Serial Bus
URL Uniform Resource Locator
VM Virtual Machine

CHAPTER 1
INTRODUCTION

1.1 Background

Overlay filesystems are pseudo-filesystems which combines multiple filesystems or multiple directories from same filesystems into one, resulting in a single directory structure that contains underlying files and sub-directories from all sources.

OverlayFS is one such filesystem implementation for Linux. Overlay filesystems are used by virtualization softwares like Docker, or used in liveboot to allow user modifications to an otherwise immutable base system, or used during system recovery.

Redox is a Unix-like general-purpose microkernel-based [1] operating system written in Rust, aiming to bring the innovations of Rust to a modern microkernel, a full set of programs, and be a complete alternative to Linux and BSD. It was inspired by various works like Plan 9, Minix, seL4, BSD, and Linux. Unlike monolithic kernels, RedoxOS implements most system services as user-space processes that communicate through a scheme-based I/O system. This architecture provides several advantages, including fault isolation, easier debugging, and enhanced security through Rust's memory safety guarantees. The operating system's filesystem layer is implemented as schemes, where each filesystem type operates as a separate service, providing a clean abstraction for different storage mechanisms.

1.2 Gap Identification

Currently, RedoxOS lacks support for overlay filesystems, which are crucial for modern containerization, live systems, and package management. While traditional filesystems [2] like ext4 and FAT are supported through the scheme system, there is no implementation of a union filesystem that can combine multiple layers into a single coherent view. This prevents RedoxOS from supporting containerization efficiently, implementing live CD/USB systems with persistent changes, or providing atomic package updates. The absence of overlay filesystem support represents a sig-

nificant gap in RedoxOS's filesystem capabilities compared to other modern operating systems.

1.3 Motivation

Overlay filesystems have become essential components in modern operating systems, serving critical roles in containerization, system deployment, and package management. Container technologies like Docker rely heavily on overlay filesystems to implement efficient image layering, where a base read-only image can be shared among multiple containers with individual writable layers for modifications. Live operating systems use overlay filesystems to provide the illusion of a writable system while running from read-only media, with changes stored in RAM or on writable storage. Additionally, overlay filesystems enable atomic system updates, snapshot capabilities, and efficient storage utilization. For RedoxOS to compete with mainstream operating systems and support modern deployment patterns, implementing overlay filesystem support is crucial.

1.4 Objectives

The primary objectives of this project are:

- To create a overlay filesystem combining an writable upper-level filesystem and read-only lower-level filesystem that are merged into a single file system Scheme.
- To develop a whiteout filesystem for tracking deletions.
- To implement a file elevation mechanism like Copy-On-Write to handle modification of files in the lower level.

CHAPTER 2

RELATED THEORY

2.1 Redox OS

2.1.1 Introduction

RedoxOS is a Unix-like operating system written in Rust, designed from the ground up with a focus on security, reliability, and correctness. Development began in 2015 with the goal of leveraging Rust's memory safety guarantees to create a more secure operating system while maintaining Unix-like familiarity and compatibility where practical.

The operating system adopts a microkernel architecture, departing from the monolithic kernel design used by traditional Unix systems like Linux and BSD. This architectural choice reflects RedoxOS's emphasis on fault isolation, modularity, and security. By moving most operating system services out of the kernel and into user space, RedoxOS minimizes the trusted computing base and limits the impact of potential vulnerabilities.

2.2 Design

2.2.1 Microkernel

Redox OS is a microkernel based operating system meaning its kernel is small and provides only the bare minimum of what is necessary. Anything that can be implemented in user-space is implemented in user-space or is in the process of being moved from the kernel-space to user-space. Hence, the kernel is only responsible for acting as a medium for communication and segregation of processes and providing minimal abstraction over the hardware.

This features of microkernel makes it more stable compared to monolithic kernels. A bug in one of the drivers of a monolithic kernel can crash the whole system. This doesn't happen in microkernel based operating system like Redox OS. Microkernel are

also better at avoiding security issues as the drivers only get minimal set of access that are required to make them functional instead of full access as the drivers in monolithic kernel. Microkernels are modular by nature allowing users to add, remove, modify, replace system components and drivers without changing the kernel.

Some well known examples of microkernel include:

- MINIX
- QNX
- Mach

2.2.1.1 Advantages of Microkernel Architecture

- **Improved Stability:** Since most services run in user-space, failure of one component (e.g., a driver) does not crash the entire system. [3]
- **Enhanced Security:** Limited privileges for each service reduce the risk of system-wide vulnerabilities.
- **Modularity:** Components such as file systems and drivers can be updated or replaced independently.
- **Fault Isolation:** Errors are contained within individual services, making debugging easier.
- **Flexibility and Maintainability:** Easier to extend and maintain due to separation of concerns.

2.2.1.2 Disadvantages of Microkernel Architecture

- **Performance Overhead:** Communication between user-space services requires message passing, which can be slower than direct function calls in monolithic kernels.
- **Complex IPC Mechanisms:** Inter-process communication (IPC) can become complex and harder to optimize.
- **Higher Context Switching Cost:** Frequent switches between user-space and kernel-space can reduce efficiency.
- **Implementation Complexity:** Designing a fully functional microkernel system requires careful handling of communication and synchronization.

2.2.2 Schemes and Resouces

The scheme, which takes its name from URI schemes, identifies the type of resource, and identifies the manager daemon responsible for that resource.

Within Redox, a scheme may be thought of in a few ways. It is all of these things:

- The type of a resource, such as "file", "NVMe drive", "TCP connection", etc. (Note that these are not valid scheme names, they are just given by way of example.)
- The starting point for locating the resource, i.e. it is the root of the path to the resource, which the system can then use in establishing a connection to the resource.
- A uniquely named service that is provided by some driver or daemon program, with the full path identifying a specific resource accessed via that service.

A scheme is typically provided by a daemon. A daemon is a program that runs as a process in userspace; it is typically started by the system during boot. When the process starts, it registers with kernel using the name of the scheme that it manages.

Redox OS refers to resources using scheme-rooted paths. This allows redox OS to

- Treat various kinds of resource in a consistent manner
- Provide resource specific behaviour through a common interface
- Enable device drivers and other resouce management to communicate with each other using the same mechanism
- Allow management of names and namespaces to provide sandboxing and other security features

A scheme-rooted path are the way resources are identified on Redox. It takes the form of `/scheme/scheme-name/resouce-name` where the first "scheme" and the slashes are literal. The scheme-name is the name used by the manager daemon as well as the type of scheme. The resource-name is the specific resource of that kind. Thus, A Redox Scheme is a type of resource, a starting point for locating a resouce, and a uniquely name service provided by some driver or daemon. A resource on Redox is anything that can be referred to by a path. For example, it can be a physical device, a logical pseudodevice, a file on a filesystem, a service that has a name etc. The client program can access a resouces using the scheme-rooted path described above.

2.3 Implementation of a Scheme

A scheme is created in the root scheme and listens for requests in the event scheme.

2.3.1 Root Scheme

The `root` scheme is a special scheme provided by the kernel that acts as a container for all other schemes. The root scheme is currnely references as ":". So, when a scheme provides needs to create a scheme, it calls `File::create(":scheme-name")`. This operation returns a file descriptor representing a message passing channel between the scheme provider and the kernel. The file operations performed by clients are translated by the kernel and passed to the scheme provider as message packets which then reads and responds to using the file descriptor.

2.3.2 Event Scheme

The `event` scheme is also a special scheme provided by the kernel. This scheme allows a scheme provider or other program to listen for events occuring on a file descriptor.

2.3.3 Providing a Scheme

A program generally performs the following speps in order to provide a scheme:

- Create a scheme and obtain a file descriptor
- Open a file descriptor for each resources that are required to provide the scheme's services
- If a timer is required, open a file descriptor for it as well using the `time` scheme
- Move to a null namespace to prevent additional resources from being accessed
- Write to `event` file descriptor to register each of the file descriptor the provider will listen to.

Then, in a loop,

- Block, waiting for an event to read.
- Read the event to determine the if it is a timer event, a resource event, or a scheme event
- If it is a resource event, perform the necessary action

- If it is a scheme event, read the request packet from the scheme file descriptor and call the handler. The request packet will indicate what kind of scheme operation was requested.
  - An `open` operation includes the name of the item to be opened. The provider is free to parse the name in any way it sees fit and determine the exact resource whose access was requested. The scheme will allocate a handle for the resource with a numbered decriptor.
  - The other operations all use the handle to look up the information associated with the resource. The operation will be performed or queued to be performed.
- After all requests have been handled, loop through every handle to determine if any queued requests are not complete. A response is sent back on the scheme file descriptor.
- Set a timer if appropriate by writing to the timer file descriptor.

2.3.4 Action of the Kernel

The kernel performs the following actions in support of a scheme:

- Any special resources required by a scheme provider are accessed as file operations on other schemes. The kernel handles access to these resources just like it does for any other scheme.
- Regular file operations from user programs are converted by the kernel into request messages for schemes. The kernel maps the user program's file descriptor to a scheme and a handle ID provided during the open operation, and places them into a packet.
- If the user program performs a blocking read or write, the program is suspended.
- The kernel sends event packets on the scheme provider's event file descriptor, waking the blocked scheme provider. Each event packet indicates whether it is for the scheme or another resource, based on the file descriptor obtained during initialization.
- When the scheme provider reads from its scheme file descriptor, it receives the packets created by the kernel describing the client request and processes them accordingly.

- When the scheme provider sends a response packet, the kernel converts the response into a return value for the user program's file operation.
- When a blocking read or write completes, the user program is marked as ready to run, and the kernel places it in the run queue.

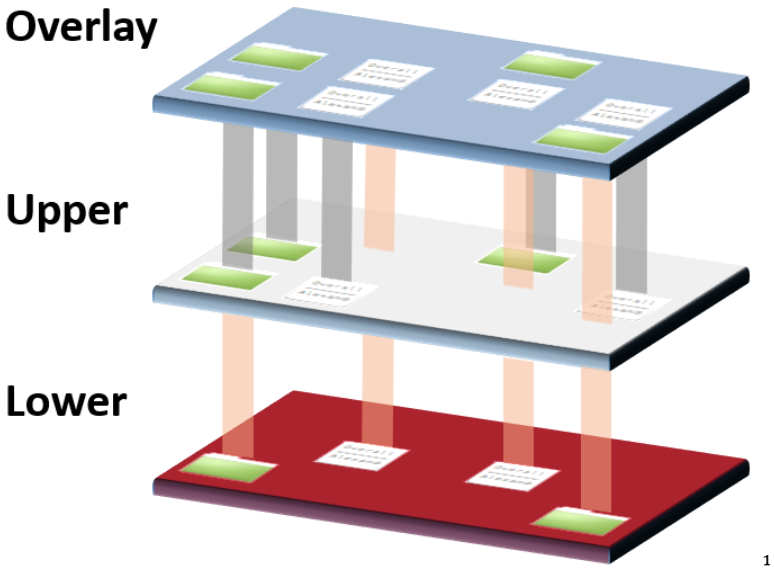
2.4 Linux's OverlayFS

OverlayFS is, as the name suggests, an overlay filesystem implementation for Linux. It combines two directory trees - an "upper" layer and a "lower" layer and presents a merged view of the both directory trees. When a name exists in both layers, the object in the upper layer shadows the object in the lower layer or, if the name refers to directories then both entires are merged. It is possible for these directorty trees to be in the same filesystem or different one. Whenever a lookup is requested in the merged directory, the lookup is performed in each actual directory and the combined result is cached in the dentry belonging to the overlay filesystem. Only the list of names from the both directories are merged and the metadata and extended attributes of only the upper layer are reported.

OverlayFS in Linux allows one, usually read-write, directory tree to be overlaid onto another, read-only directory tree. All modifications go to the upper, writable layer.

This type of mechanism is most often used for live CDs but there's a wide variety of other uses.

The implementation differs from other "union filesystem" implementations in that after a file is opened all operations go directly to the underlying, lower or upper, filesystems. This simplifies the implementation and allows native performance in these cases. [4]



[1]

Figure 1: Visual Representation of Layers in an Overlay Filesystem

2.4.1 Copy Up Operation

Overlay filesystem implements a copy up strategy to handle modifications to files originating from the read-only lower layer. When a process attempts to modify a file that exists only in the lower layer, OverlayFS performs a copy-up operation. This operation copies the entire file from the lower layer to the upper layer before applying any modifications. Subsequent reads and writes then operate on the copied version in the upper layer, leaving the original file in the lower layer unchanged.

The copy-up mechanism has important implications for performance and storage. Large files incur significant overhead during the initial modification, as the entire file must be copied regardless of how small the actual change is. This operation is atomic from the perspective of other processes, ensuring consistency during the copy-up procedure.

The copy-up mechanism is also performed before any operation that changes a file's attributes without modifying its actual content. For example, operations such as chown and chmod require a copy-up operation; only then are the actual operations performed on the copied version of the file.

2.4.2 Whiteouts and Opaque Directories

[1]Source: Embedded Computing Design, https://embeddedcomputing.com/technology/processing/understand-what-an-overlayfs-is-and-how-it-works

The lower layer of overlayFS is mostly read-only. So, to support removing files and directories without modifying the lower layer, overlayFS needs to store the record of those files and directories in the upper layer that have been removed. This is done using whiteouts and opaque directories. A whiteout is created as a character device with `0/0` device number or as zero-size regular file with the extended attribute "trusted.overlay.whiteout". When a whiteout is found in the upper layer, any matching name in the lower layer is ignored and the whiteout itself is kept hidden.

A directory is made opaque by setting the extended attribute "trusted.overlay.opaque" to "y". Opaque directory are handled similarly to whiteouts in that when one is found in the upper layer, any directory in the lower layer with the same name is ignored.

2.4.3 Readdir and Renaming

When a request is made for reading a merged directory, both the directory in the upper and lower layers are read and have their name list merged. The merging is done in the folloing way: upper is read first and then from the lower ignoring the names that already exists. OverlayFS can handle renaming of merged directory or directory on lower layer in two ways. It can either return EXDEV error or if "redirect_dir" feature is enabled, then the directory will be copied up and the "trusted.overlay.redirect" extended attribute is set to the path of the original location from the root of the overlay and finally the directory is moved to the new location.

2.4.4 Permission Model

OverlayFS stashes credentials that will be used when accessing lower or upper layers. Permission checking in the overlay filesystem follows these principles:

- Permission check should return the same result before and after the copy up operation.
- Task creating the overlay mount must not gain additional privileges.

CHAPTER 3

LITERATURE REVIEW

In virtualization, OverlayFS offers an efficient method for managing virtual machine disk images [5]. A common base image, containing the operating system and standard software, can serve as a read-only lower layer. Multiple virtual machines can then share this single base image, with each VM's individual changes, such as user files and configuration updates, being written to a unique, writable upper layer. This copy-on-write approach significantly reduces disk space consumption and accelerates the provisioning of new virtual machines, as there is no need to duplicate the entire base OS for each instance.

Containerization [6] is a primary use case for OverlayFS, as exemplified by Docker. Container images are built-in a series of read-only layers, where each layer represents a set of file system changes (e.g., installing a software package). When a container is launched, these immutable layers are stacked as the lower layers of an OverlayFS mount. A new writable upper layer is then added for the running container. All write operations performed by the container, such as creating log files or temporary data, are isolated to this upper layer. This architecture allows for the efficient sharing of common layers among multiple containers and ensures that the underlying image remains pristine, facilitating lightweight, fast, and reproducible container deployments.

OverlayFS is fundamental to the functionality of modern Live USB operating systems. The core OS files are stored on the USB device as a compressed, read-only filesystem, which serves as the lower layer. When the system boots, an OverlayFS is created, using a writable upper layer that resides in the system's RAM. This allows the user to interact with the system as if it were installed on a hard drive—installing software, creating files, and changing settings. Because the upper layer is non-persistent and stored in memory, all modifications are discarded upon reboot, ensuring that the Live USB returns to its original, clean state for every session.

In system rescue and recovery environments, OverlayFS provides a flexible and safe workspace. A rescue system is typically booted from a read-only medium like a CD-

ROM or a write-protected USB drive. This read-only system image acts as the lower layer. By mounting an overlay with RAM-backed upper layer, an administrator can write files, download repair tools, and modify configurations in a temporary, writable filesystem. This prevents any accidental or permanent changes to the core rescue tools on the boot medium, guaranteeing the integrity and reliability of the recovery environment for subsequent use.

The Linux kernel's OverlayFS implementation [4], documented in the kernel documentation [7], provides the foundational reference for overlay filesystem design. Linux OverlayFS supports multiple lower layers, implements sophisticated whiteout mechanisms using special filetypes (DT_WHT), and handles complex scenarios like opaque directories and metadata-only updates. The Linux implementation uses extended attributes (xattrs) for storing overlay-specific metadata, though this approach may need adaptation for RedoxOS due to limited xattr support in existing RedoxOS filesystems.

Docker extensively uses overlay filesystems for implementing container image layers. Docker's approach demonstrates practical overlay filesystem usage patterns, including how to handle large numbers of layers, optimization strategies for common operations, and integration with container runtime systems. Docker's experience highlights the importance of efficient copy-on-write mechanisms and the challenges of managing complex layer hierarchies.

BSD union mounts [8] introduced early concepts of layering directories but faced challenges related to complexity and stability when handling write operations and whiteouts. Later, the AUFS (AnotherUnionFS) aimed to provide a comprehensive union filesystem with numerous features; however, it was ultimately rejected from the mainline Linux kernel due to its dense, complex, and hard-to-maintain codebase. The UnionFS project from Stony Brook University made significant strides by formalizing the use of whiteout concepts to handle file deletions elegantly. These foundational ideas regarding layer composition and whiteouts heavily influenced and were carried forward into the design of OverlayFS.

Copy-On-Write (COW) is a broader concept utilized across various computing domains to optimize resource usage by delaying the copying of data until a modification is requested. In storage systems, Btrfs and ZFS implement block-level COW, where only the modified data blocks are written to new locations, preserving the old blocks for snapshots. In contrast, OverlayFS utilizes a file-level COW approach. As discussed by Mizusawa et al. (2018) [6] in their IEEE SMARTCOMP paper examining OverlayFS container performance, the file-level COW architecture is simpler to implement and manage within a union mount context. However, it introduces a notable overhead for large files because modifying even a single byte requires copying the entire file from the lower layer to the upper layer before the write can proceed.

Filesystem implementation differs significantly between microkernel and monolithic kernel architectures. In monolithic kernels like Linux, filesystems reside within kernel space and directly interact with block devices, leveraging shared kernel structures. Conversely, microkernels run filesystem drivers in isolated userspace processes. For instance, the Minix filesystem operates entirely in userspace, communicating with the core kernel via message passing. Similarly, GNU Hurd translators act as userspace servers that handle filesystem operations, providing a conceptual parallel to RedoxOS schemes. FUSE (Filesystem in Userspace) on Linux offers related functionality but is often constrained by the performance characteristics and context-switching overhead incurred when transitioning repeatedly between kernel space and userspace.

OverlayFS has become a critical component in modern production systems. Crucially, Docker adopted OverlayFS as its default storage driver (starting with Linux kernel 3.18+), cementing its role in containerization. Furthermore, the Kubernetes Container Runtime Interface (CRI) relies heavily on overlay semantics for efficient pod storage and image management. Beyond container runtimes, systems like OSTree and Nix employ overlay semantics to enable powerful atomic system updates, allowing entire operating system deployments or package trees to be swapped safely without breaking running applications.

CHAPTER 4

METHODOLOGY

This chapter outlines the approach taken in designing, implementing and testing of the overlay filesystem for RedoxOS. The methodology encompasses the entire process undertaken in this major project from choosing a development model to technical approaches taken during implementation as well as testing strategies used in evaluating and improving the filesystem.

4.1 Development Approach

The project followed a iterative prototyping software development model. Under this model, we developed an initial prototype upon which we continuously added features until the final product met the objectives we initially set out to meet. Each feature addition went through the design, implementation, testing and refining phases. All of the was preceded by an initial and flexible design of the overall system.

Implementing a filesystem or any other service provider for RedoxOS involves implementing a predefined set of functions, all of which are disguished as filesystem operation. Following our iterative approach, we first developed a prototype that implemented only the basic of these operations that made the prototype functional. These operations included `open`, `close`, `read`, 'write' and `getdents`. With these operations implemented, the overlay filesystem could function as a very simplified filesystem. We implemented the other operations like `unlink`, `rmdir`, `dup`, `fchmod` etc iteratively.

The major reason for adpoting the iterative approach is to make the process of testing easier. The RedoxOS is not self-hostable yet. This means developing for RedoxOS while using RedoxOS is not possible and we had to use other development environment, namely linux, for the development of the overlay filesystem. For the purposes of testing, this meant we had to implement the feature inside of our development environment in linux and build the filesystem with additional changes along with the RedoxOS image and run the RedoxOS and test it within the RedoxOS. Seperating each operation in an iteration help isolate the problems with an implementation

of operation within that iteration. This helped catch bugs easily and evaluate each functionality seperately.

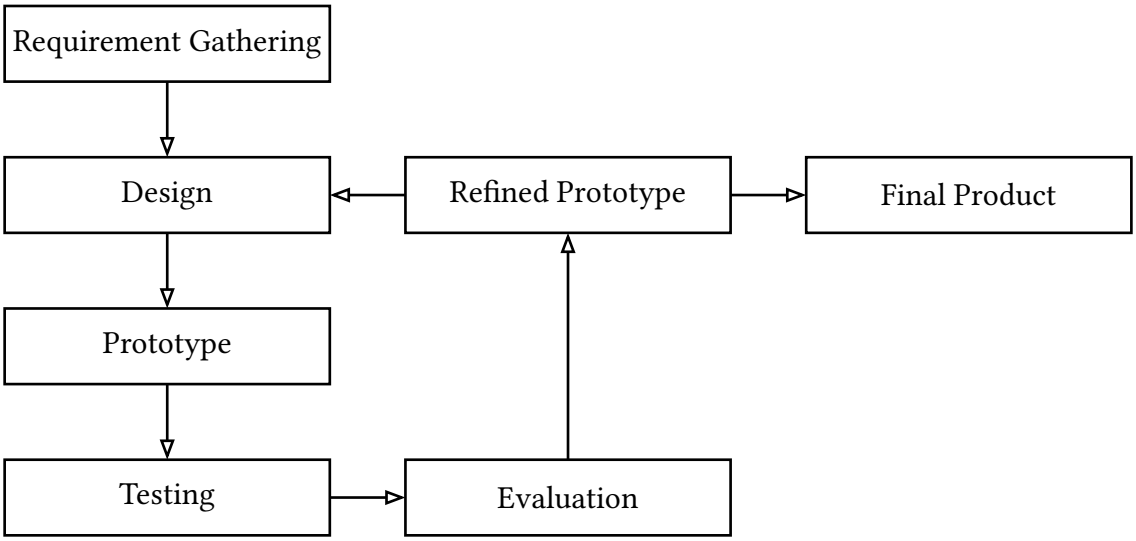


Figure 2: Prototype Model

4.2 Requirement Analysis and Designing the Filesystem

The project started with a comprehensive study of the RedoxOS architecture, specifically the study of Resouces and Schemes, as well as the study of linux's implementation of overlay filesystem. This study proved essential when making design choices for our filesystem and its integration with RedoxOS.

4.2.1 Study of the Scheme Based Architecture

We conducted an in-depth analysis of RedoxOS's scheme based architecture. This involved analyzing the existing scheme implementations in the RedoxOS codebase, particularly the `file` scheme as well as reading up on the available documentation on the topic. This helped further our understanding of the scheme architecture, its interface contracts, data flow patterns, and integration points. The scheme trait defines a set of methods that any resource provider must implement, including operations for opening, reading, writing, seeking, and obtaining metadata about resources. Understanding these requirements was critical for designing an overlay filesystem that properly integrates with RedoxOS's resource abstraction model. We examined how existing filesystem schemes handle path resolution, permission checking, and error propagation. This analysis revealed important conventions and patterns used

throughout RedoxOS that our implementation must follow for consistency and compatibility.

4.2.2 Overlay Filesystem Analysis

We also investigated existing overlay filesystem implementations to understand common design patterns, challenges, and solutions. We studied Linux OverlayFS implementation details, Docker's use of overlay filesystems for container images, and academic literature on union filesystem designs. This study identified key technical challenges that our implementation must address, including copy-on-write semantics for file modifications, whiteout file mechanisms for representing deletions in read-only layers, and directory merging algorithms. Understanding these challenges informed our architectural decisions and helped us anticipate implementation difficulties.

4.3 Implementation

We adopted a delegate pattern for our filesystem implementation where we wrote the core of our filesystem without any RedoxOS specific functionality beside a `FileSystem` structure. This core utilizes the standard library of the Rust programming language as well as the an implementation of C standard library referred to as `libc`. We then wrote a thin wrapper around this core to provide the scheme implementation specific to RedoxOS. The motivation for this seperation of concern is the same as the one described above for choosing the iterative prototyping. Due to the complicated process of setting up the RedoxOS image and testing the implementation inside it, even the smallest amount of change would have incurred significant amount of testing time. This delegate pattern helped us reduce this time by allowing us to test the core logic of each operation for correctness inside our development environment.

Each instance of our filesystem scheme implementation is a mounted overlay filesystem. The central component of this filesystem is a structure aptly named `FileSystem`. This structure stores the path of the upper and lower layer directory trees as well as the name under which the specific instance is registered. In addition to this, the major component of this structure is the `files` field which is a mapping from an

unsigned integer to resources the filesytem holds. When a resource is requested on the filesystem through `open` system call, if the requested resource is present and the requesting client has appropriate permission, the resource data is stored and mapped to a unique number. This number is then returned to the requesting client. The client can use this number to request operation like read, write, etc on the resource.

4.4 Testing

Testing was one of the core focus of our implementation for the first day. Testing enabled us to ensure our implementation for correctness, reliability and compatibility.

4.4.1 Unit Testing

As described above, we used delegate pattern to make it easier for testing core functionality. We used a rust crate called `tempdir` to simulate the lower and upper layer and test the functionality like successful opening of files, copy on write mechanism, whiteout and opaque directories. For this, we wrote numerous test cases that test each core functionality our filesystem provides.

4.4.2 Functional Testing

Functional tests validate end-to-end filesystem behavior from an application perspective. These tests perform filesystem operations through standard interfaces and verify expected outcomes, mimicking how actual applications would use the overlay filesystem.

Test scenarios cover common use cases such as creating files in the overlay, modifying files from lower layers, deleting files and verifying whiteout behavior, and directory operations across layers. We also test edge cases including name conflicts between layers, permission interactions, and resource exhaustion conditions.

4.4.3 Compatibility Testing

To ensure that our overlay filesystem integrates properly with the broader RedoxOS ecosystem, we perform compatibility testing with existing RedoxOS utilities and applications. This includes testing with standard file manipulation tools (cp, mv, rm,

ls), text editors and file viewers, and build systems that perform complex filesystem operations.

Compatibility testing identifies any deviations from expected filesystem behavior that might break existing software, even if the overlay filesystem technically meets specification requirements.

4.5 Project Timeline

The following Gantt chart illustrates our planned project timeline from initial research through final presentation. The timeline is divided into three major phases: Research Phase, Development Phase, and Final Phase.

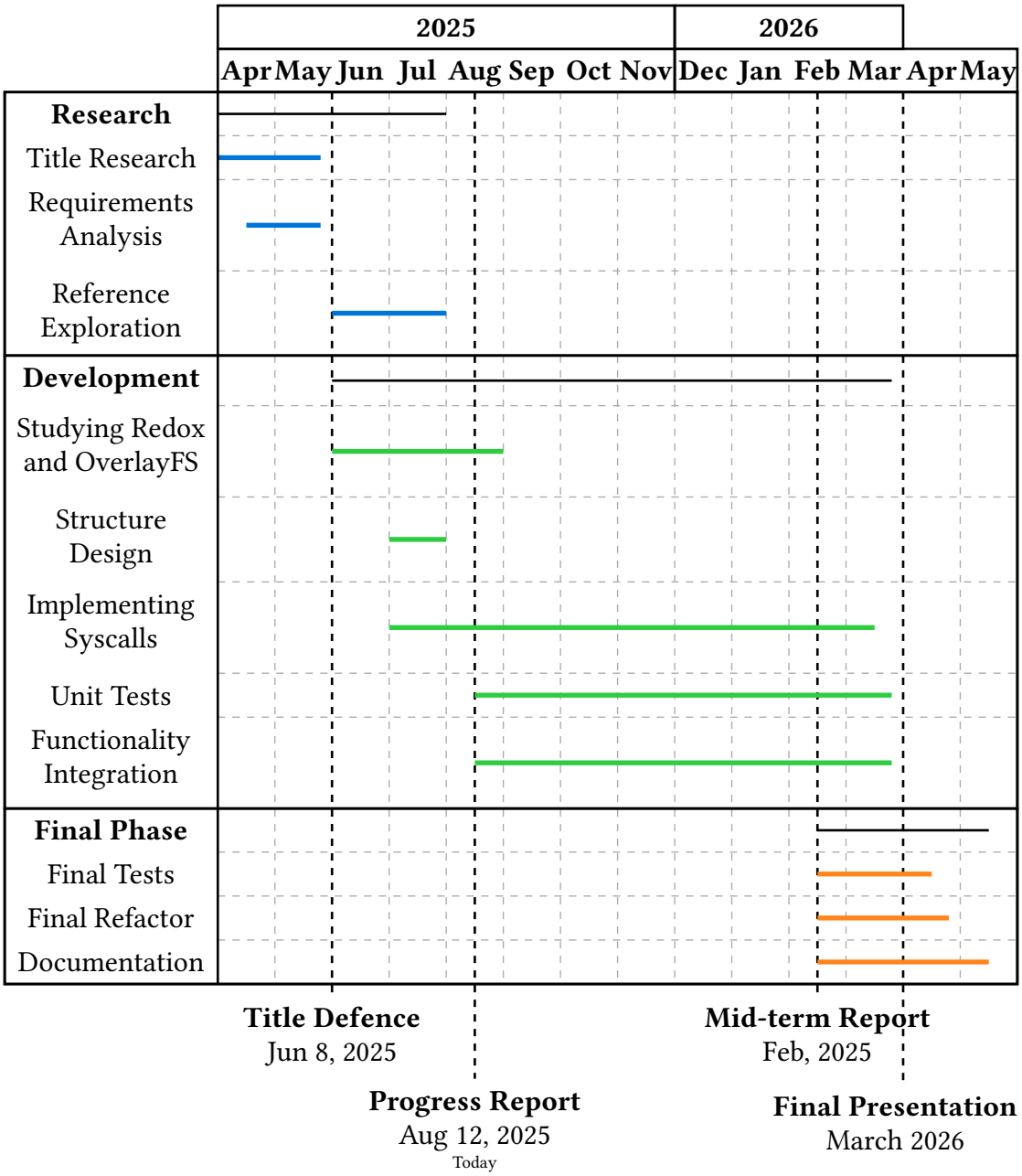


Figure 3: Project Timeline

CHAPTER 5

IMPLEMENTATION

5.1 Overall Architecture

The system is designed around a two-layer delegate pattern, which effectively isolates the operating system's scheme interface from the core filesystem logic. The `OverlayScheme` structure, which implements the `SchemeSync` trait, handles RedoxOS-specific concerns such as translating system calls into filesystem operations. This structure wraps the `FileSystem` component, a platform-independent core that contains the actual overlay logic. This architectural separation proved critical, as it allowed the majority of the filesystem logic to be developed, tested, and debugged on Linux before undergoing the full RedoxOS image rebuild cycle.

The process flow can be visualized as follows:

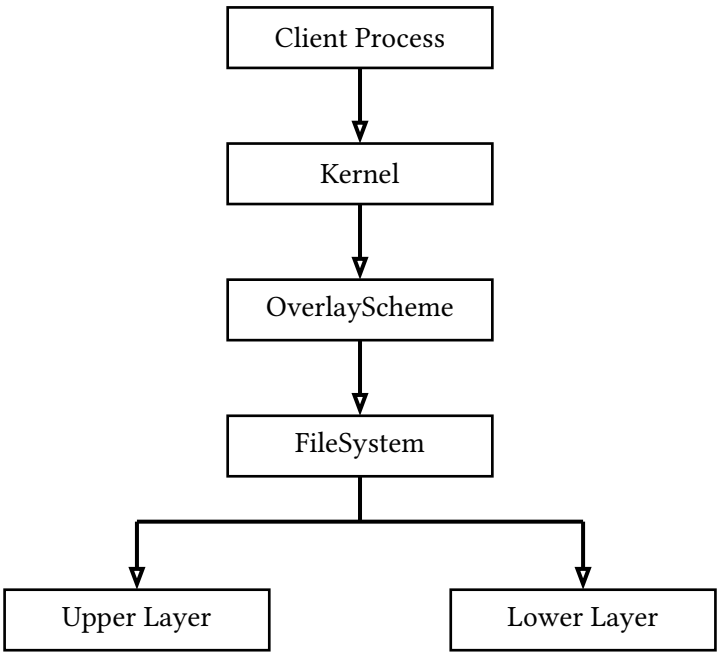


Figure 4: Client process call hierarchy

5.2 Core Data Structures

The overlay filesystem relies on several core data structures to manage state and track resources. The primary structure is the `FileSystem` struct, which contains the following critical fields:

- `upper`: A path referencing the directory used as the writable upper layer.
- `lower`: A path referencing the directory used as the read-only lower layer.
- `name`: A string identifier for the mounted scheme.
- `files`: A `HashMap<usize, FileHandle>` that maps unique file descriptors to their corresponding internal file handles.

To track the state of individual open files or directories, the system uses the `FileHandle` struct, which includes:

- `path`: The relative path of the file within the overlay.
- `layer`: An enumeration (`enum`) indicating whether the file was opened from the `Upper`, `Lower`, or represents a `New` file entirely.
- `flags`: The operational flags (such as read, write, or append) passed during the initial open system call.
- `data`: A memory buffer used to store directory listing results, ensuring consistency during repeated `getdents` calls over the same directory.

5.3 Path Resolution

Path resolution is the mechanism by which the overlay determines the origin layer of a requested file. The resolution algorithm follows a strict step-by-step process:

1. Construct the upper path by joining the requested relative path with the upper layer root directory.
2. Check for the existence of a `.wh.` whiteout marker corresponding to the requested filename in the upper directory.
3. If a whiteout marker exists, immediately return `ENOENT` to indicate the file does not exist, shadowing any underlying file.
4. If the file exists in the upper layer without a whiteout, resolve the path to the upper layer and return successfully.
5. If the file is not found in the upper layer, check the lower layer. Let the path resolve if it exists there.

6. If the file is absent in both layers, return `ENOENT`.

Additionally, during directory lookups, the system checks for opaque directory markers (`.wh.__<dirname>`). If a directory is opaque, path resolution for its contents skips the lower layer entirely.

5.4 Copy-On-Write Implementation

Copy-On-Write (COW) ensures that the lower layer remains truly read-only while enabling write capabilities. When a write operation targets a file originating from the lower layer, the system performs a copy-up procedure. This process is executed in the following steps:

1. Determine the source lower path and the destination upper path.
2. Create any missing intermediate directories leading to the final upper path to ensure the directory structure exists.
3. Open the source lower layer file exclusively for reading.
4. Create a temporary file in the same upper directory as the final destination.
5. Copy all file data from the lower layer to the temporary upper file in memory chunks.
6. Atomically rename the temporary file to the final upper path.
7. Update the internal `FileHandle` layer field to indicate that the file now resides in the `Upper` layer.

The atomic rename operation guarantees that a partially written file is never visible at the final path. If the system were to power off or crash during the copying phase, the temporary file might remain orphaned, but the final file view would either completely point to the unmodified lower layer file or the fully copied upper layer file, preventing data corruption.

5.5 Whiteout Implementation

File deletions in a union mount require recording the removal of lower-layer files without actually erasing them from the read-only media. To accomplish this, the system implements a whiteout mechanism utilizing a specific naming conven-

tion: `.wh.<filename>` for standard files and `.wh.__<dirname>` for directory tracking (opaque directories).

In the Linux kernel's OverlayFS, special character device files are commonly used for whiteouts. However, RedoxOS filesystems do not universally or reliably support special device file types across all backends. Therefore, the implementation utilizes regular zero-byte files as whiteout markers instead.

Crucially, all `.wh.*` entries are filtered out of `getdents` results. Applications accessing the filesystem via the scheme interface are entirely unaware of these metadata entries, providing a seamless illusion of a unified and modifiable filesystem.

5.6 Directory Merge Algorithm

When an application attempts to list the contents of a directory that spans both the upper and lower layers, the overlay must merge the contents. The directory merge algorithm proceeds as follows:

1. Read all entries from the upper layer directory. Skip any entries beginning with `.wh.`. Add the remaining valid entries to the combined result set and a `seen-names` tracking set.
2. Collect all whiteout names from the upper layer by stripping the `.wh.` prefix from their filenames. Add these exact original names to a separate whiteout set.
3. If the directory is not marked as opaque, proceed to read the corresponding lower layer directory entries. While iterating, skip any name that is already present in the `seen-names` set or in the whiteout set. Add the remainder to the combined result set.
4. Return the final, deduplicated, and unified combined result set to the caller.

CHAPTER 6

RESULTS

This chapter presents the outcomes of the overlay filesystem implementation for RedoxOS. It covers the final state of the implemented system, the behaviour of each component under testing, and observations made throughout development. Where relevant, areas for future improvement are noted.

6.1 Implemented System

The final implementation consists of a fully functional overlay filesystem scheme for RedoxOS. The system merges an upper writable layer with a lower read-only layer and presents them to applications as a single unified filesystem, accessible through a RedoxOS scheme path. The implementation is written entirely in Rust and integrates with RedoxOS through the SchemeSync trait.

The system is structured around two principal components. The OverlayScheme structure handles scheme-level concerns — translating between the operating system's scheme interface and the filesystem logic. The FileSystem structure underneath it contains the actual overlay logic, including layer resolution, handle tracking, copy-on-write, and whiteout management. This separation proved effective throughout development, allowing most of the logic to be developed and tested on Linux before being built into a RedoxOS image.

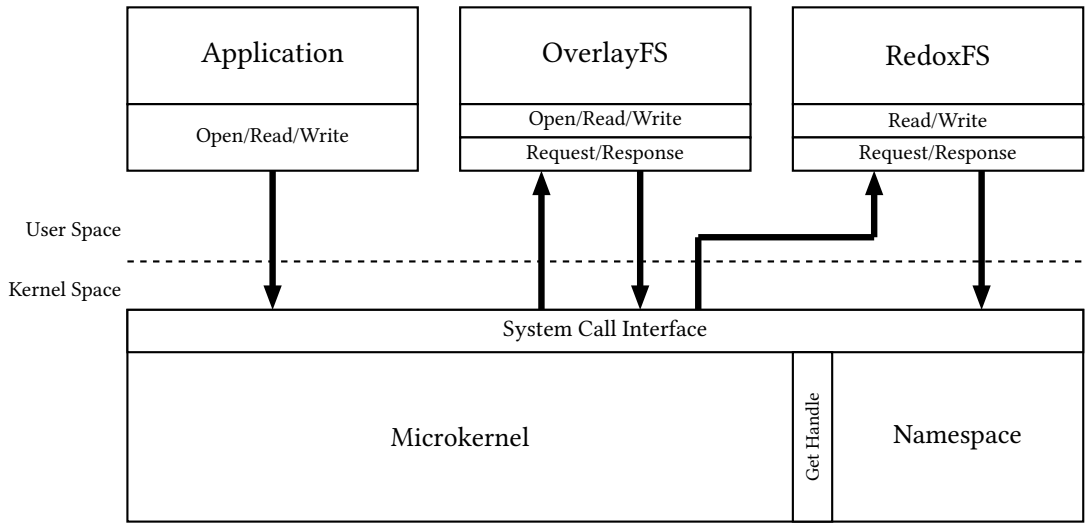


Figure 5: Architecture diagram

6.2 File Handle Management

When a client opens a resource through the overlay scheme, the FileSystem assigns the resource a unique integer handle and stores the associated state — including which layer the file was resolved from and any buffered data — in an internal map. This handle is returned to the client and subsequently used for all further operations (reads, writes, metadata queries, etc.). On close, the handle and its associated resources are released.

This design maps naturally onto RedoxOS's scheme model, where the scheme provider is responsible for tracking open resources on behalf of clients.

6.3 Implemented System Calls

The following system calls have been fully implemented and validated through unit testing and integration testing within RedoxOS.

6.3.1 open, openat

Path resolution proceeds by checking the upper layer first. If the file is found there, it is opened directly. If not, the lower layer is checked. Whiteout entries in the upper layer are respected during this lookup — a whiteout for a given name causes the

corresponding lower layer entry to be treated as nonexistent. User and group IDs from the calling context are forwarded to the underlying layer for permission checking.

6.3.2 read and write

Read operations are dispatched to whichever layer the file was resolved from at open time. Write operations implement copy-on-write: if the target file originates from the lower layer, the file is first copied in its entirety to the upper layer before any modification is made. All subsequent writes then proceed against the upper layer copy. This ensures the lower layer remains unmodified at all times.

6.3.3 close

Closing a file descriptor releases the corresponding handle from the internal map and frees associated resources. Files pending copy-up that are closed before any write is issued do not trigger the copy-up operation.

6.3.4 getdents

Directory enumeration merges the contents of both layers. The upper layer is read first, producing an initial set of names. The lower layer is then read, with names already present in the result set — or shadowed by whiteout entries — excluded. The final result presents a unified directory listing to the caller. Opaque directories in the upper layer cause their lower layer counterparts to be ignored entirely.

Iteration state across multiple getdents calls is maintained using opaque offsets, consistent with the behaviour expected by RedoxOS utilities.

```
admin@overlay:/ $ ls
.
..
common_dir
lower_dir
lower_file.txt
read-only.txt
upper_dir
upper_file.txt
admin@overlay:/ $ ls lower_dir
.
..
inner.txt
admin@overlay:/ $ ls upper_dir
.
..
inner.txt
admin@overlay:/ $ ls common_dir
.
..
common.txt
admin@overlay:/ $ cat common_dir/
common.txt
I'm in Upper Layer on common_dir.

Overlay in Action!
admin@overlay:/ $
```

Listing 1: Merged/Overlayed view as seen by the `getdents` syscall.

```
~/Projects/overlayfs/overlay
master*
❯ tree -a
.
├── lower
│   ├── common_dir
│   │   └── common.txt
│   ├── lower_dir
│   │   └── inner.txt
│   ├── lower_file.txt
│   └── read-only.txt
├── upper
│   ├── common_dir
│   │   └── common.txt
│   ├── upper_dir
│   │   └── inner.txt
│   └── upper_file.txt
└── work

8 directories, 7 files

~/Projects/overlayfs/overlay
master*
❯ \cat upper/common_dir/common.txt
I'm in Upper Layer on common_dir.

Overlay in Action!

~/Projects/overlayfs/overlay
master*
❯ \cat lower/common_dir/common.txt
I'm in Lower Layer on common_dir.
```

Listing 2: Actual Underlying Filesystem

6.3.5 unlink and rmdir

Deleting a file that exists only in the upper layer removes it directly. For files that exist in the lower layer, a whiteout entry is created in the upper layer to record the deletion without modifying the lower layer. rmdir includes validation that a directory is empty before deletion, and creates the appropriate marker in the upper layer when the directory being removed originates from the lower layer.

6.3.6 fstat, fstatvfs, fsize

Metadata queries return information from the layer in which the file resides. For upper layer files this is straightforward. For lower layer files, metadata is retrieved from the lower layer. fstatvfs aggregates filesystem statistics from both layers to report accurate overall capacity and inode usage figures.

6.3.7 fchmod and fchown

Permission and ownership changes on lower layer files trigger a copy-up before the modification is applied. The change is then made to the upper layer copy. This ensures that the permission model is consistent before and after copy-up, matching the behaviour described in Linux's OverlayFS documentation.

6.3.8 ftruncate and futimens

Truncation and timestamp modification follow the same copy-on-write pattern. If the target file resides in the lower layer, a copy-up is performed first. The operation is then applied to the upper layer copy.

6.3.9 flink

Hard link creation is supported within the overlay. Both the source and destination must reside in the upper layer; creating links to lower layer files triggers copy-up first.

6.3.10 fcntl, fevent, fpath

File descriptor flag manipulation, event registration for asynchronous I/O, and path retrieval are all implemented and function as expected through the scheme interface.

6.4 Copy-On-Write Behaviour

Copy-on-write is one of the central mechanisms of the overlay filesystem. The implementation copies the entire file from the lower layer to the upper layer atomically before applying any modification. This mirrors the approach used by Linux OverlayFS and ensures that partial writes do not leave files in an inconsistent state visible to other processes.

The copy-up operation is triggered by writes, truncations, permission changes, and ownership changes. It is not triggered by reads or metadata queries, keeping read paths efficient for workloads that only read from lower layer files.

One known limitation of this approach, inherited from the design itself, is that copy-up of large files incurs cost proportional to the file size even when only a small portion of the file is being modified. This is an inherent trade-off of the copy-on-write strategy and is consistent with how other overlay filesystem implementations behave.

```
admin@overlay:/ $ ls
.
..
common_dir
lower_dir
lower_file.txt
read-only.txt
upper_dir
upper_file.txt
admin@overlay:/ $ ls lower_dir
.
..
inner.txt
admin@overlay:/ $ cat lower_dir/
inner.txt
I'm in Lower Layer on lower_dir.

My filename is inner.txt
admin@overlay:/ $ echo
"Modification!" > lower_dir/
inner.txt
admin@overlay:/ $ cat lower_dir/
inner.txt
"Modification!"
admin@overlay:/ $
```

Listing 3: COW as seen by the `getdents` syscall.

```
~/Projects/overlayfs/overlay
master* 33s
❯ tree -a
.
├── lower
│   ├── common_dir
│   │   └── common.txt
│   ├── lower_dir
│   │   └── inner.txt
│   ├── lower_file.txt
│   └── read-only.txt
├── upper
│   ├── common_dir
│   │   └── common.txt
│   ├── lower_dir
│   │   └── inner.txt
│   ├── upper_dir
│   │   └── inner.txt
│   └── upper_file.txt
└── work

9 directories, 8 files

~/Projects/overlayfs/overlay
master*
❯ \cat lower/lower_dir/inner.txt
I'm in Lower Layer on lower_dir.

My filename is inner.txt

~/Projects/overlayfs/overlay
master*
❯ \cat upper/lower_dir/inner.txt
"Modification!"
```

Listing 4: Actual Underlying Filesystem after COW

6.5 Whiteout Handling

Whiteout files are created as regular files in the upper layer at the path corresponding to the deleted name. When path resolution encounters a whiteout, the corresponding name in the lower layer is suppressed. Whiteout entries are not exposed to applications — they are filtered from getdents results and do not appear as regular files through the scheme interface.

```
admn@overlay:/ $ ls
.
..
common_dir
lower_dir
lower_file.txt
read-only.txt
upper_dir
upper_file.txt
admin@overlay:/ $ ls lower_dir
.
..
inner.txt
admin@overlay:/ $ cat lower_dir/
inner.txt
I'm in Lower Layer on lower_dir.

My filename is inner.txt
admin@overlay:/ $ echo
"Modification!" > lower_dir/
inner.txt
admin@overlay:/ $ cat lower_dir/
inner.txt
"Modification!"
admin@overlay:/ $
```

Listing 5: Removing of File as seen by the `getdents` syscall.

```
~/Projects/overlayfs/overlay
master*
❯ tree -a
.
├── lower
│   ├── common_dir
│   │   └── common.txt
│   ├── lower_dir
│   │   └── inner.txt
│   ├── lower_file.txt
│   └── read-only.txt
├── upper
│   ├── common_dir
│   │   └── common.txt
│   ├── lower_dir
│   │   └── inner.txt
│   ├── upper_dir
│   │   └── inner.txt
│   └── upper_file.txt
└── work

9 directories, 8 files
```

Listing 6: Actual Underlying Filesystem after Deletion of File

6.6 Opaque Directories Handling

Opaque directories are handled during directory merging. When an upper layer directory is marked opaque, the lower layer directory of the same name is excluded from the merge entirely, as if it did not exist. This is used when a directory has been renamed or recreated in a way that should not inherit contents from the lower layer.

```
admin@overlay:/ $ rm common_dir/
common.txt
admin@overlay:/ $ rmdir common_dir
admin@overlay:/ $ ls
.
..
lower_dir
lower_file.txt
upper_dir
upper_file.txt
admin@overlay:/ $
```

Listing 7: Removing Directory as seen by the `getdents` syscall.

```
~/Projects/overlayfs/overlay
master*
❯ tree -a
.
├── lower
│   ├── common_dir
│   │   └── common.txt
│   ├── lower_dir
│   │   └── inner.txt
│   ├── lower_file.txt
│   └── read-only.txt
├── upper
│   ├── lower_dir
│   │   └── inner.txt
│   ├── upper_dir
│   │   └── inner.txt
│   ├── upper_file.txt
│   ├── .wh.__common_dir
│   └── .wh.read-only.txt
└── work

8 directories, 9 files
```

Listing 8: Actual Underlying Filesystem after Removing Directory

6.7 Testing

6.7.1 Unit Tests

Unit tests exercise the core overlay logic in isolation using temporary directories on Linux to simulate upper and lower layers. The tempdir crate is used to create and clean up test environments automatically. Tests cover the following:

- Successful file opens from both layers with correct precedence
- Copy-on-write triggering and execution, including verification that the lower layer is unmodified after a write
- Whiteout creation on deletion and suppression of shadowed names during lookup and enumeration
- Opaque directory behaviour during merge
- Directory merge correctness including deduplication and whiteout filtering
- Error conditions including missing files, permission failures, and resource exhaustion

All unit tests pass consistently on the development platform.

6.7.2 Integration Tests

Integration testing was conducted inside a RedoxOS virtual machine. Tests were performed using standard RedoxOS utilities operating on the mounted overlay scheme. The following scenarios were validated:

- Creating new files in the overlay and reading them back
- Reading files present only in the lower layer
- Modifying a lower layer file and verifying copy-on-write execution
- Deleting a lower layer file and verifying the whiteout prevents subsequent access
- Listing a merged directory and confirming correct unification of both layers
- Applying chmod and chown to lower layer files and verifying copy-up

All of the above scenarios produced correct results. The overlay was also exercised with the RedoxOS cp, ls, rm, and cat utilities without unexpected behaviour.

6.8 Identified Issues and Limitations

Several issues identified during testing are worth noting:

- Directory enumeration with large directories spanning both layers performs redundant stat calls that could be reduced with caching. This does not affect correctness but impacts performance for large directory trees.
- Some permission edge cases arise when a file exists in both layers with different permission bits. The current implementation gives precedence to the upper layer's permissions after copy-up, which is correct, but intermediate states during copy-up are not fully protected against concurrent access.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

The implementation detailed in this report successfully meets all three core objectives established in Chapter 1. A unified overlay scheme was developed for RedoxOS, successfully combining a writable upper layer with a read-only lower layer into a single view. The system effectively tracks file deletions using a robust whiteout-based approach and correctly manages file elevation via the copy-on-write mechanism. Standard RedoxOS utilities, including `ls`, `cp`, `rm`, and `cat`, operate seamlessly through the overlay scheme without requiring any modification, verifying the system's transparency and correctness.

The architectural decision to employ a delegate pattern separating the filesystem logic from the `SchemeSync` trait implementation proved to be highly valuable. This decoupling enabled rapid development and debugging in a Linux environment without incurring the full RedoxOS image rebuild cycle for every iteration. Furthermore, leveraging Rust's ownership and borrowing system successfully eliminated common systems programming vulnerabilities, such as use-after-free errors on file handles and data races on the handle map, entirely at compile time. Ultimately, this implementation fills a previously missing gap in the RedoxOS ecosystem, mitigating roadblocks that previously hindered containerization, live system operation, and atomic update workflows.

7.2 Future Work

Multiple Lower Layer Support The current implementation supports merging a single lower layer with the upper layer. However, enabling Docker-style image stacking necessitates support for an ordered list of lower layers. To integrate this successfully, the path resolution and directory merge algorithms would need to iterate

appropriately over multiple lower paths in priority order, resolving files and metadata from the highest available underlying layer before proceeding downwards.

Rename with Redirect Directory Support Currently, the filesystem returns an `EXDEV` error when attempting a cross-layer directory rename. In contrast, the Linux OverlayFS manages this using the `trusted.overlay.redirect` extended attribute (`xattr`) to track redirected directories. Implementing this capability requires comprehensive `xattr` support within RedoxFS, an enhancement currently planned for future versions of the operating system.

Lazy Copy-On-Write The existing copy-on-write behavior replicates an entire file from the lower layer to the upper layer even for a single-byte modification. A lazy approach would be exceptionally beneficial, copying only the specific regions or data blocks that are actively written. This optimization significantly reduces the processing and storage cost associated with large files but does require implementing considerably more complex atomicity guarantees to maintain consistency.

Concurrent Access Protection During Copy-Up Presently, a potential race condition exists where a process could read a partially copied file through a separate handle during the copy-up phase. To eliminate this issue and ensure robust concurrency protections, establishing a per-file read/write lock around the specific copy-up procedure would adequately address this vulnerability.

Directory Enumeration Caching The primary identified performance bottleneck involves redundant `stat` calls executed during `getdents` operations over large directories that span both layers. Employing a short-lived cache of the merged listing would drastically reduce the number of underlying filesystem calls required, notably improving directory traversal performance without negatively affecting overarching system correctness or accuracy.

Integration with a Container Runtime Looking ahead, performing end-to-end validation of the overlay scheme as the primary storage backend for a lightweight container runtime deployed directly on RedoxOS would serve as the ultimate demonstration of the implementation's correctness and practical viability.

Extended Attribute Based Whiteouts Once RedoxFS successfully gains comprehensive `xattr` support, migrating the existing whiteout system from the distinct `.wh.` naming convention to `xattr`-based whiteouts would vastly improve interoperability. This transition aligns better with tools that inherently understand the Linux OverlayFS metadata format and fully eliminates problematic edge cases where legitimate user filenames happen to begin with `.wh.`.

REFERENCES

[1] Ribbon, "Microkernels." [Online]. Available: https://doc.redox-os.org/book/microkernels.html

[2] J. Soller and Ribbon, "RedoxFS." [Online]. Available: https://doc.redox-os.org/book/redoxfs.html

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 5th ed. Harlow, United Kingdom: Pearson Education Limited, 2024, p. 66.

[4] M. Szeredi, "Overlay Filesystem." [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=e9be9d5e76e34872f0c37d72e25bc27fe9e2c54c

[5] J. Scholz, "Using OverlayFS to Save Disk Space." [Online]. Available: https://e2b.dev/blog/scaling-firecracker-using-overlayfs-to-save-disk-space

[6] N. Mizusawa, J. Kon, Y. Seki, J. Tao, and S. Yamaguchi, "Performance Improvement of File Operations on OverlayFS for Containers," in *2018 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2018, pp. 297–302. doi: 10.1109/SMARTCOMP.2018.00019.

[7] N. Brown, "Overlay Filesystem." [Online]. Available: https://kernel.org/doc/html/latest/filesystems/overlayfs.html

[8] J.-S. Pendry and M. K. McKusick, "Union Mounts in 4.4BSD-Lite," in *Proceedings of the USENIX Technical Conference on UNIX and Advanced Computing Systems*, Dec. 1995, pp. 25–33.